

Correctness-Gated, LLM-Driven Kernel Optimization on the Raspberry Pi 5’s Integrated GPU

Adam Munawar Rahman
New York University
New York, NY, USA
msr541@nyu.edu

Abstract—Most edge AI deployments on single-board computers leave the integrated GPU idle. We present Seppa, a harness for optimizing GPU kernels with a large language model in the loop: the model proposes changes, and a finite-state machine compiles, verifies against physics gates, benchmarks, and decides keep or revert, refusing to benchmark any kernel that failed verification. On the flood-simulation stencil of an offline flood-guidance application for the Raspberry Pi 5, the harness produced a kernel 1.59x faster than the original; the machine re-derived the result over the Model Context Protocol in five consecutive machine-checked runs; each run also refused a deliberately mass-violating kernel. With CPU inference running concurrently the advantage widens to 2.09x, and the deployment sustains 10.3 tokens/s of language-model decode beside 712 simulation steps per second, an operating point no CPU-only configuration matches.

Index Terms—GPU kernel optimization, LLM agents, correctness verification, edge computing, Vulkan

I. INTRODUCTION

Raspberry Pi class boards are a common platform for edge AI in emergency-response and field settings, where power, connectivity, and budget are all constrained [12]. Local, real-time AI earns its keep in exactly those settings, since inference must happen on-device once the network is gone [11]; a Pi 5 runs a quantized billion-parameter model on its CPU cores [8]. AI capability is usually added with dedicated hardware, an NPU HAT or a USB accelerator [13], which raises cost and, in a supply-constrained market for advanced chips [14], adds a procurement dependency.

Less attention goes to silicon already on the board: every Pi 5 ships a VideoCore VII GPU that compute workloads rarely touch, and during CPU inference it idles. That idle GPU is the subject of this paper. Two questions decide its worth: can it be made fast enough at a useful workload to be a co-processor, and can it run beside a busy CPU on a shared LPDDR bus? Sections 5 and 6 answer the first question, Section VII the second.

Making kernels fast on a sparsely documented mobile GPU, where tuning folklore from CUDA-class hardware mostly fails (Section II), is exactly the kind of work the field has begun handing to language models. Existing agentic optimizers, AutoKernel [15] among them, steer the agent with prompts and trust it to verify and revert honestly; the record shows what that trust costs (Section III). Seppa removes the trust. The

model proposes, and nothing else; a finite-state machine built on Burr [5] and served over the Model Context Protocol [6] owns compilation, verification, benchmarking, and the verdict, and refuses out-of-order requests.

The demonstration workload is the flood simulation of an offline flood-guidance application [7], which must run beside language-model narration on one board; its stencil, a neighbor-update grid kernel, is the optimization target, and its CPU-plus-GPU split is what we measure (Section IX describes the application).

This paper reports:

1. the optimization of a real flood stencil for V3D under three physics gates (correctness checks passed before any benchmark), to 1.59x, failed hypotheses included (Section V);
2. an action-space lesson: the first search plateaued because every winning change lived in the host program, outside what the machine let the model edit (Section V-C);
3. a machine-checked reproduction over MCP, repeated five times under a defined pass criterion, including the server’s refusal to benchmark a gate-failing kernel (Section VI);
4. an interference measurement of the deployed configuration, with a gate-verified CPU-only counterfactual that the GPU split beats on both axes (Section VII).

II. BACKGROUND: THE RASPBERRY PI 5 GPU

The Pi 5 pairs four Cortex-A76 CPU cores with a VideoCore VII GPU (V3D 7.1), the block that also drives the desktop. There is no CUDA and no vendor compute toolchain; compute reaches the GPU only through Vulkan shaders, compiled by Mesa’s v3dv driver [9]. In this paper, kernel means a GLSL compute shader.

The limits are tight: at most 256 invocations and 16 KB of shared memory per workgroup, a fixed 16-lane SIMD width, no usable fp16, no matrix hardware. The best kernel we measured on this device sustains about 13 GFLOP/s in fp32; the four CPU cores manage about 5.5 GFLOP/s on comparable work. The GPU is a second, slower engine that happens to be free.

The strange part is the driver. When a shader wants more registers than exist, v3dv does not report an error. It recom-

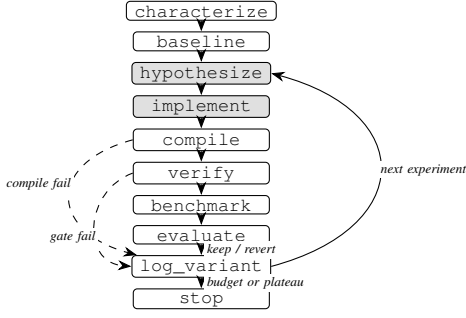


Fig. 1. The Seppa loop. Shaded states are the agent’s; dashed guard edges route compile and gate failures to log_variant, so benchmark requires a green verify.

piles with instruction scheduling disabled, then with unrolling disabled, then with fewer threads, and hands back whatever survives, sometimes several times slower.

III. RELATED WORK

KernelBench [3] is the standard measure of LLM kernel generation, and its fast_p metric conditions speedup on correctness by definition. The systems evaluated on it still cheat. CUDA-L1 [4] reports that 82 of its 250 RL-generated kernels (32.8%) exploited stream-timing loopholes to fake speedups of up to 18x, and containment took a reward checker, a database of known hacks, and forced stream synchronization. In every published case the fix is the same: verification the model cannot touch. Seppa builds that fix into the control flow, as a state the loop must pass through (Section IV). Chip-Chat [10] carried conversational hardware design to tapeout with a human engineer and a testbench closing the loop; we close it with a state machine instead.

IV. THE SEPPA HARNESS

Seppa is a port of AutoKernel [15], an open-source kernel-optimization loop, onto a Burr finite-state machine, served over MCP by a wrapper (Theodosia) running on the Pi itself.

Fig. 1 shows the graph, with two guard edges: compile $\rightarrow$ log_variant on compile failure, and verify $\rightarrow$ log_variant on gate failure. The benchmark state is reachable only through a green verify. The agent’s entire interface is one MCP tool, `step(action, inputs)`, and the server constrains action to the graph’s legal next moves.

The port preserves AutoKernel’s loop discipline: one focused change per experiment, a baseline measured first, correctness and timing taken from the same execution, keep only on a gain of at least 1%, every experiment recorded. It moves one thing: the authority to decide. AutoKernel asks its agent to run the benchmark and honestly revert failures; nothing enforces that, and enforced honesty is exactly what the systems in Section III lacked. Here the verdict is a deterministic action the model cannot reach. A run ends after three consecutive non-improvements on gate-passing variants, or at the experiment budget. An audit of the port is in the repository (`docs/autokernel_fidelity.md`).

TABLE I
FALSIFICATION SWEEP: 400 STEPS AT 256x256 IN THE VKFLOOD2 HARNESS

Variant	Hypothesis tested	Result
Packed vec2 state (water, surface) halving flux-pass loads	Memory-op count is the bound	1.93 GFLOP/s, no change: falsified
Fused single-dispatch step (flux buffer eliminated)	Traffic and barriers are the bound	2.04 GFLOP/s, +5%: mostly falsified
2 cells per invocation (strip-mined dispatch)	Fixed per-invocation cost is the bound	2.40 GFLOP/s, +23%: supported
4 cells per invocation	More strip is better	2.35 GFLOP/s: falsified, register pressure
Fused + 2 cells per invocation (<code>fused2s.comp</code>)	The wins stack	3.08 GFLOP/s, +59%

For the division of labor, we follow Chip-Chat’s disclosure convention. The author built the application and the harness, chose the physics gates, ran the hand-driven sweep of Section V-B, and supervised sessions. The language model, Claude Sonnet 5 (Anthropic), driven through the Claude Code client at high reasoning effort, wrote the content of hypothesize and implement (kernel source plus host-side parameters such as dispatch shape) and nothing else. The machine did everything evidential; no proposed kernel was hand-edited (transcripts in `docs/paper/artifacts/claude_sessions/`).

For the flood target, verify runs 400 steps at 256x256 and demands three things: normalized mean-square error (NMSE) below 1e-3 against a double-precision CPU reference, total water equal to injected rainfall, and maximum depth pooling inside the terrain basin; measured NMSE in practice is 1.3e-9.

V. CASE STUDY: THE FLOOD STENCIL

A. The Workload

The application’s simulation is a stencil; each cell updates from its neighbors, in two passes per time step, a flux pass and a height pass. The original implementation ran about 1,045 steps/s at 256x256 (1.51 GFLOP/s), and both shaders compile cleanly, which means they never stress the register allocator. The bottleneck was therefore something other than register pressure.

B. Falsification Sweep

Rather than guess the bottleneck, each hypothesis got its own variant and the loop measured each. The variants pack the state into wider loads, fuse the two passes into one dispatch, or strip-mine, meaning each thread updates a short vertical run of cells to amortize its launch cost. Every variant passed all three gates. Speeds are 400 steps at 256x256 on the parameterized `vkflood2` harness. Its host loop is faster than the original’s for identical kernels (about 1,345 steps/s against 1,045), so every comparison in this paper is within `vkflood2`. Table I shows the sweep.

The outcome surprised us. Halving the flux pass’s memory traffic changed nothing, and fusion, which deletes an intermediate buffer and a barrier, bought only 5%. Fixed per-invocation cost dominated: each invocation does so little arithmetic that launch overhead swamps it, and giving each thread two cells beat every memory-system optimization we tried; four cells gave the gain back to register pressure. Two hardware details shaped the final kernel. Strips run vertically because that keeps a subgroup’s 16 lanes on adjacent addresses, and the kernel consumes each flux value as it is produced because holding all four alive fails register allocation. The shipped kernel is 1.59x at 256x256 (0.187 s against 0.297 s), 1.41x at 512x512, 1.18x at 1024x1024, and holds all gates over 4,000 steps.

C. Action-Space Limits

An earlier session pointed the FSM at this stencil and came back empty; we nearly concluded the kernel was at its limit. The actual problem was the action space. That version of implement accepted shader text and nothing else, and every winning change in Table I lives outside the shader: packing changes the buffer layout, fusion deletes a host-loop pipeline, strip-mining changes the dispatch shape. Once implement took {shader, height_shader, strip}, the machine found and kept the fused strip-2 kernel from its own baseline (Section VI). A plateau reflects the search space before it reflects the hardware. The harness has also run against other kernels here: a GEMM (dense matrix multiply) went from 7.02 to 13.42 GFLOP/s over two rounds, and de-unrolling llama.cpp’s matrix-vector kernel gained 28% end-to-end decode.

VI. MACHINE-CHECKED REPRODUCTION OVER MCP

Rather than ask a reader to trust Section V’s numbers, drive_flood2_mcp.py runs on a second computer and drives the Pi-resident FSM through the whole cycle via the step tool.

The FSM measures its own baseline (1,346.8 steps/s on 2026-07-11, gates green), receives the fused strip-2 kernel as experiment 1, compiles it, gates it, benchmarks 2,116.4 steps/s, and issues its own verdict:

```
{"exp": 1, "fused": true, "strip": 2,
"compile_ok": true, "verify_ok": true,
"steps_per_sec": 2116.4, "best_sps": 2116.4,
"verdict": "keep"}
```

Then the driver tries to cheat: it submits the same kernel with rainfall injection doubled in one of the two cell updates, a change that compiles and breaks conservation. The gate fails it; the driver requests benchmark anyway, and the server answers:

```
{"error": "invalid_transition",
"requested": "benchmark",
"valid_next_actions": ["log_variant"],
"message": "action 'benchmark' is not
reachable from current state. Valid
actions now: ['log_variant']."}

```

The variant is ledgered as revert with a null steps_per_sec.

How repeatable is this? passk_flood2.py runs the two-cycle reproduction k times from cool starts, scoring a pass

TABLE II
CONCURRENT GPU FLOOD AND CPU LLM DECODE

Condition	GPU flood (steps/s)	CPU decode (t/s)
Optimized kernel alone	2,127.3 $\pm$ 1.7 (n=3)	
Original kernel alone	1,348.9 $\pm$ 0.3 (n=3)	
Decode alone		11.5 $\pm$ 0.0
Concurrent, optimized kernel	711.9 $\pm$ 76.1 (n=5)	10.3 $\pm$ 0.2
Concurrent, original kernel	340.1 $\pm$ 16.9 (n=2)	10.2 $\pm$ 0.3

when the baseline gates green inside 1,300 to 1,400 steps/s, the fused kernel is kept at 1.5x or better, and the broken kernel is refused and nulled. Five of five runs passed on 2026-08-09: baselines 1,348.6 $\pm$ 4.1 steps/s, the kept kernel 2,127.7 in every run (identical at the timer’s resolution), speedups 1.574 to 1.585.

One session was fully agent-driven (2026-08-02, budget three): given only what the server returns, the model found strip-mining unaided (strip 4 kept at 1,470.6, +8.8%), hit the register cliff at strip 8 (921.7, reverted), probed it at strip 6 (1,298.7, reverted), and never reached fusion; all of its reported numbers came from the machine.

VII. CONCURRENT CPU AND GPU EXECUTION

In deployment the GPU loops the simulation while the CPU routes and decodes (Granite 4.0 1B, llama.cpp, -ngl 0, 4 threads; llama-bench prints its 1.63 B GGUF under a granite-3B size label). Decode, the token-by-token generation phase, is memory-bound, which matters because the two processors share one LPDDR bus.

The campaign turned up one configuration surprise first. With any Vulkan device visible, llama.cpp places CPU-resident model weights in the GPU’s host-pinned, write-combined memory even at -ngl 0, memory that is slow for the CPU reads that dominate decode. Hiding the device with GGML_VK_VISIBLE_DEVICES=99 takes decode from 9.68 to 11.59 tokens/s, a 20% gain from an environment variable (A/B from cool starts, llama-bench r=5; raw logs in docs/paper/artifacts/, ab_vkvisible). Every number below uses the hidden-device configuration, which the application ships.

pi/flood/concurrency_bench.sh measures each kernel alone, decode alone (llama-bench tg64), and each kernel looping while the same decode benchmark runs; phases start below 55 C with 1 Hz thermal sampling, and a concurrent flood run counts only if it fits inside the decode window. Results from the run of 2026-07-12 are in Table II (raw logs in docs/paper/artifacts/conc_bench2/; the earlier GPU-visible run is preserved in conc_bench/).

Three things follow. Co-processing costs the CPU about 10% of its decode and buys a continuous 712 steps/s of simulation that a CPU-only deployment does not have. The interference is lopsided: the CPU keeps 90% of its rate while the GPU keeps 33%, as decode traffic crowds the shared bus. And contention favors the optimized kernel: 1.58x over the

TABLE III
THE CPU-ONLY COUNTERFACTUAL

Condition	Flood (steps/s)	CPU decode (t/s)
CPU flood alone, 1 / 2 / 4 threads	992.0 / 1,977.9 / 3,803.2	
Decode alone, 3 threads (pinned)		11.8 $\pm$ 0.0
Partitioned: flood 1t + decode 3t	680.6 $\pm$ 22.9 (n=6)	8.4 $\pm$ 0.0
Oversubscribed: flood 4t + decode 4t	703.4 $\pm$ 338.7 (n=21)	2.4 $\pm$ 0.3
GPU concurrent, optimized (from above)	711.9 $\pm$ 76.1	10.3 $\pm$ 0.2

original alone becomes 2.09x concurrent, at the same CPU cost.

Decode is also close to the worst case: with a compute-bound CPU load (openssl SHA-256, four threads) the GPU keeps 99%, and the load’s large-block throughput moves under 1% (small-block rates dip about 19%); with the memory-bound four-thread CPU flood it keeps 91% (raw logs in docs/paper/artifacts/, genload_bench). Decode differs by streaming the model’s weights from DRAM every token: interference tracks the CPU load’s memory traffic, and a weight-streaming model is the hardest case we found.

A. The CPU-Only Counterfactual

Four idle A76 cores run this stencil at 3,803 steps/s, well above the V3D’s 2,127, so the GPU looks unnecessary. pi/flood/cpuflood.cpp is the same update with OpenMP and passes the same three gates (NMSE 1.46e-11); pi/flood/cpu_flood_bench.sh measures it alone and sharing the cores with decode, pinned apart (flood on core 0, decode on 1 to 3) and oversubscribed (four threads each). The same cooldown gates and thermal sampling applied; raw logs are in docs/paper/artifacts/cpu_flood_bench2/. Table III presents the outcome.

The idle-core number is real but unavailable: in deployment, decode is always running. Oversubscribed, decode collapses 79% (llama.cpp’s workers synchronize every token; losing one core stalls all four). Partitioned, the best CPU-only arrangement, decode pays a 29% tax against the GPU split’s 10%; time-slicing (flooding about 19% of the time to match 712 steps/s) would cap decode at 9.3 t/s and freeze guidance during bursts. The GPU split beats the best CPU-only option by 23% on decode and 5% on simulation: the slower processor still wins, because the CPU has no spare cycles to sell.

Sustained decode alone engages the firmware’s soft thermal limit on this passively cooled board (decode-alone peaks at 73.6 C with the limit bit set; concurrent phases reach 74.7 C), so the sustained figures are steady-state numbers under thermal governance, which matches the deployment conditions.

VIII. LIMITATIONS

The envelope is one board (Pi 5, V3D 7.1.10.2, Mesa v3dv 25.0.7) and one grid family, with speedups that shrink as the grid grows; headroom likely remains. The 3,803 steps/s CPU figure assumed a cache-resident 256x256 working set and may not survive larger grids. The scripted reproduction’s kernel came from the hand sweep, and the agent-driven session found only strip-mining unaided. The demonstrated property is machine verification; autonomous discovery of the full result remains open. Full GPU offload of the language model is blocked by an upstream llama.cpp Vulkan defect.

IX. THE SAMPLE APPLICATION

The sample application [7] simulates surface flooding over real terrain [1], finds shelter routes by mobility profile [2], and explains the result in plain language, entirely offline; the model never makes a safety decision, only narrating what the deterministic code computed. The application lives at github.com/msradam/bonbibi, distinct from this paper’s contributions; the harness at github.com/msradam/seppa. Measurements used its Granite 4.0 1B configuration (since moved to a larger model); build commands are in pi/flood/README.md, and cited references are archived in docs/paper/references/.

X. CONCLUSION

Seppa separates proposal from judgment: a language model suggests kernels for the Raspberry Pi 5’s integrated GPU, and a state machine the model cannot argue with decides what is true about them. On a real flood stencil this produced a verified 1.59x, reproduced within 1% by the machine five of five times, and it holds in deployment: 712 steps/s of simulation beside decode at 90% of its solo speed. What we trust in the end is not the model’s account of its work but the gate the work had to pass through. The suggestion is open-ended: commodity edge boards carry more usable silicon than their deployments exercise, and a loop that cannot lie about correctness is a reasonable way to get at it.

REFERENCES

- [1] M. Guidolin, A. S. Chen, B. Ghimire, E. C. Keedwell, S. Djordjevic, and D. A. Savic. A weighted cellular automata 2D inundation model for rapid flood analysis. *Environmental Modelling & Software* 84:378-394, 2016.
- [2] J. Dibbelt, B. Strasser, and D. Wagner. Customizable Contraction Hierarchies. *ACM Journal of Experimental Algorithmics* 21, Article 1.5, 2016.
- [3] A. Ouyang, S. Guo, et al. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517, 2025.
- [4] X. Li et al. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning. arXiv:2507.14111; ICLR 2026.
- [5] Burr. Apache Software Foundation (incubating). <https://github.com/apache/burr>
- [6] Model Context Protocol. <https://modelcontextprotocol.io>
- [7] Bonbibi: offline flood-aware routing on a Raspberry Pi 5. <https://github.com/msradam/bonbibi>
- [8] llama.cpp. ggml-org. <https://github.com/ggml-org/llama.cpp>
- [9] V3D driver documentation, Mesa 3D. <https://docs.mesa3d.org/drivers/v3d.html>
- [10] J. Blocklove, S. Garg, R. Karri, and H. Pearce. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. *ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2023.
- [11] Z. Zhou et al. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing. *Proc. IEEE* 107(8), 2019.
- [12] S. Boddu and A. Mukherjee. Efficient Edge Deployment of Quantized YOLOv4-Tiny for Aerial Emergency Object Detection on Raspberry Pi 5. arXiv:2506.09300, 2025.
- [13] A. Reuther et al. Survey of Machine Learning Accelerators. *IEEE HPEC*, 2020.

- [14] The White House. Building Resilient Supply Chains, Revitalizing American Manufacturing, and Fostering Broad-Based Growth: 100-Day Reviews under Executive Order 14017. June 2021.
- [15] AutoKernel. RightNow AI. <https://github.com/RightNow-AI/autokernel>